

EcoExchange Microservices - Single-Command Executable Launcher Architecture

Project: EcoExchange Capstone Platform

Module: runner-service

Tech Stack: Java 21 / Spring Boot 4.1.0 / Spring Cloud 2025.1.2

1. Executive Summary & Architecture Overview

EcoExchange is an enterprise cloud-native B2B digital marketplace designed to enable industrial waste producers and recyclers to trade scrap materials and calculate carbon emissions offset. The platform consists of **9 decoupled microservices**:

- `config-service` (Port 8888): Centralized Git/Native property management server.
- `discovery-service` (Port 8761): Netflix Eureka Naming Server for service registration.
- `apigateway` (Port 8080): Spring Cloud Gateway routing client traffic to downstream microservices.
- `identity-service` (Port 8081): Security, JWT token issuance, and organization verification.
- `marketplace-service` (Port 8082): Catalog listings, material properties, and bidding/offer engine.
- `order-service` (Port 8083): Order checkout, transaction processing, and shipping tracking.
- `search-service` (Port 8084): High-performance search, auto-suggestions, and catalog filters.
- `notification-service` (Port 8085): Kafka consumer for real-time user alerts and emails.
- `analytics-service` (Port 8086): Calculation engine for waste tonnage diverted and CO2 emissions saved.

2. Technical Challenge & Architectural Rationale

[!WARNING]

Running 9 separate terminal commands (`mvn spring-boot:run`) creates high developer friction, excessive memory overhead, and error-prone startup ordering.

Attempts to combine all 9 services into a single Spring Boot application class (`SpringApplication.run`) fail due to three core architectural constraints:

- Embedded Server Port Collisions:** Each microservice starts an embedded Tomcat or Netty server. Running them in a single JVM context attempts to bind multiple web servers to the same port, causing `BindException`.
- Spring Bean Definition Overrides:** Component scanning across all 9 modules imports duplicate beans, configuration classes, and security filters into a single context, breaking dependency injection.

3. **Strict Startup Dependencies:** Microservices require the **Config Service** (8888) to fetch properties and the **Discovery Service** (8761) to register endpoints before initializing their REST controllers.

3. Maven POM Architecture & Reactor Setup

A. Root Aggregator POM (pom.xml)

```
<?xml version="1.0" encoding="UTF-8"?>

4.0.0

<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.0</version>
    <relativePath/>
</parent>
<groupId>com.industry-connect</groupId>
<artifactId>ecoexchange</artifactId>
<version>0.0.1-SNAPSHOT</version>
<packaging>pom</packaging>
<name>ecoexchange</name>
<description>EcoExchange Microservices Platform</description>
<properties>
    <java.version>21</java.version>
    <spring-cloud.version>2025.1.2</spring-cloud.version>
</properties>
<modules>
    <module>config-service</module>
    <module>discovery-service</module>
    <module>apigateway</module>
    <module>identity-service</module>
    <module>marketplace-service</module>
    <module>order-service</module>
    <module>search-service</module>
    <module>notification-service</module>
    <module>analytics-service</module>
    <module>runner-service</module>
</modules>
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-dependencies</artifactId>
            <version>${spring-cloud.version}</version>
            <type>pom</type>
```

```
        <scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>
</project>
```

B. Launcher Module POM (runner-service/pom.xml)

```
<?xml version="1.0" encoding="UTF-8"?>

    4.0.0
    <parent>
        <groupId>com.industry-connect</groupId>
        <artifactId>ecoexchange</artifactId>
        <version>0.0.1-SNAPSHOT</version>
        <relativePath>../pom.xml</relativePath>
    </parent>
    <groupId>com.industry-connect</groupId>
    <artifactId>runner-service</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>runner-service</name>
    <description>Common Multi-Service Process Launcher for EcoExchange Platform</description>
    <properties>
        <java.version>21</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter</artifactId>
        </dependency>
    </dependencies>
    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration>
                    <mainClass>com.industry_connect.runner.EcoExchangeRunner</mainClass>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

4. The Solution: Process Launcher Pattern (runner-service)

The EcoExchange repository addresses this challenge using a dedicated launcher module: `runner-service`. Instead of running services in a shared JVM classloader, `runner-service` uses Java's native `ProcessBuilder` API to spawn each service in its own isolated Java runtime.

Microservice Execution Matrix

Phase	Service Name	Port	Launch Delay	Execution Mode
Phase 1	<code>config-service</code>	8888	4.0 seconds	Synchronous Process
Phase 2	<code>discovery-service</code>	8761	6.0 seconds	Synchronous Process
Phase 3	<code>apigateway</code>	8080	Parallel	ThreadPool Executor
Phase 3	<code>identity-service</code>	8081	Parallel	ThreadPool Executor
Phase 3	<code>marketplace-service</code>	8082	Parallel	ThreadPool Executor
Phase 3	<code>order-service</code>	8083	Parallel	ThreadPool Executor
Phase 3	<code>search-service</code>	8084	Parallel	ThreadPool Executor
Phase 3	<code>notification-service</code>	8085	Parallel	ThreadPool Executor
Phase 3	<code>analytics-service</code>	8086	Parallel	ThreadPool Executor

5. Source Code Walkthrough

Implementation in `EcoExchangeRunner.java`:

```

package com.industry_connect.runner;

import java.io.File;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class EcoExchangeRunner {
    private static final Logger log = LoggerFactory.getLogger(EcoExchangeRunner.class);

    public static void main(String[] args) {
        File rootDir = new File(".").getAbsoluteFile();

        // Step 1: Launch Config Service (Port 8888)
        log.info(">>> Launching Config Service (Port 8888)...");
        startServiceJar(rootDir, "config-service");
        sleep(4000);

        // Step 2: Launch Discovery Service (Port 8761)
        log.info(">>> Launching Discovery Service (Port 8761)...");
        startServiceJar(rootDir, "discovery-service");
        sleep(6000);

        // Step 3: Launch remaining 7 microservices concurrently
        ExecutorService executor = Executors.newFixedThreadPool(7);
        String[] remainingServices = new String[]{
            "apigateway", "identity-service", "marketplace-service",
            "order-service", "search-service", "notification-service", "analytics-service"
        };

        for (String serviceName : remainingServices) {
            executor.submit(() -> startServiceJar(rootDir, serviceName));
        }
        executor.shutdown();
    }

    private static void startServiceJar(File rootDir, String serviceName) {

```

```

        try {
            File targetDir = new File(rootDir, serviceName + File.separator + "target");

            File[] jarFiles = targetDir.listFiles((dir, name) -> name.endsWith(".jar") && !name.endsWith(".original"));

            List<String> command = new ArrayList<>();
            command.add("java");
            command.add("-jar");
            command.add(jarFiles[0].getAbsolutePath());

            ProcessBuilder pb = new ProcessBuilder(command);
            pb.directory(rootDir);
            pb.inheritIO(); // Streams logs directly to parent terminal console
            pb.start();
        } catch (Exception e) {
            log.error("Error starting service: " + serviceName, e);
        }
    }
}

```

6. Step-by-Step Execution Guide

Step 1: Release Active Port Bindings

Ensure no background Java processes are holding active microservice ports:

```

# Windows (PowerShell / CMD)
taskkill /F /IM java.exe

# Linux / macOS
pkill -f java

```

Step 2: Build & Package Project Reactor

Run the Maven package command from the root project folder:

```

mvn clean package -DskipTests

```


Step 3: Single-Command Execution

Execute the common launcher JAR file:

```
java -jar runner-service/target/runner-service-0.0.1-SNAPSHOT.jar
```

[!NOTE]

Result: *All 9 microservices launch sequentially and concurrently, streaming real-time operational logs directly to your single active console!*